

Data Structure (week 3)

講師：陳俊豪

補充 1 :

str

不只是基礎元素

```
str1 = 'Hello'  
str1 += ' World'
```

ordered 也是有序的 !!

```
str1[0]  
str1[2:6]
```

可 loop !!

```
for each in str1:  
    print(each)
```

In-class exercises 1

1. 用 for 迴圈寫出九九乘法表, 從 $1 \times 1 = 1$ 到 $9 \times 9 = 81$, 結果需要如下圖:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
2 x 1 = 2
2 x 2 = 4
```

...

```
8 x 9 = 72
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

In-class exercises 2

```
ls1 = [1, 2, 3, 4]
for i in range(len(ls1)):
    p = ls1.pop()
    ls1.append(p)
print(ls1)
```

>>> Ans ?

>>>

```
ls1 = 'admin'
for char in ls1:
    if char == 'm':
        continue
    print(char)
```

>>> Ans ?

>>>

```
var = 10
while var:
    var -= 1
    if var > 5:
        continue
    print(var)
```

>>> Ans ?

>>>



In-class exercises 3

1. 給定 `list1 = [10, 20, [300, 400, [5000, 6000], 500], 30, 40]`, 請問如何使用索引 (slice) 拿到整數 6000 ?
2. 寫一個使用 `if-elif-else` 與 `input` 的程式, 程式功能要能夠判斷 `input` 的值是不是 5 的倍數, 如果是就 `print 'Yes!'`, 否則就 `'No!'`。

參考 `data = input('Please input something :')`

In-class exercises 4

```
{ [ ( int, float, str, bool ) ] }
```

 $+, -, *, /$

+ *****

1. 寫一個程式實現以下結果(結果一樣就全對):



← 8個*

2. 寫一個程式實現右方結果(結果一樣就全對):

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Python 基礎重點介紹

函數 function

函數 function

- 函數 function 是為了讓程式更結構化與重覆利用的程式設計方式。

函數定義規則

```
def func1(參數1, 參數2):
```

```
    ans = 參數1 + 參數2
```

```
return ans
```

使用 def 來做定義，後面接函數名稱，再接參數設定。

函數有分成有返回值 (return) 與無返回值，一般建議使用不要使用 global。

函數 function

- 函數 function 是為了讓程式更結構化的程式設計方式。

函數定義規則

```
def func1(參數1, 參數2):  
  
    ans = 參數1 + 參數2  
  
return ans
```

參數還可以設定預設值, ex.
參數1=10, 這樣若在呼叫階段沒有代值則會以此為主。

函數 function

- 函數 function 是為了讓程式更結構化的程式設計方式。

函數使用方式

```
par1 = 10  
  
par2 = 20  
  
ans = func1(par1, par2)
```

直接來寫一個加法器吧！

- 定義完成後可以直接呼叫，把參數都代入即可。
- 參數位置一般為按位置代入，單也可以直接寫清楚「關鍵字」以避免錯誤發生。

func1(par2=10, par1=5)

函數 function

- 有 return vs 無 return, 函數不一定都需要 return。
- 回傳 None 表示無 return。

```
def func1(參數1, 參數2):
```

```
    ans = 參數1 + 參數2
```

```
    print(ans)
```



```
ans = func1(10, 20)
```

```
# None
```

```
func1(10, 20)
```

函數 function

- 資料處理過程經常不需要回傳值。
- 會傳入需要處理的資料，在函數內直接進行修改。

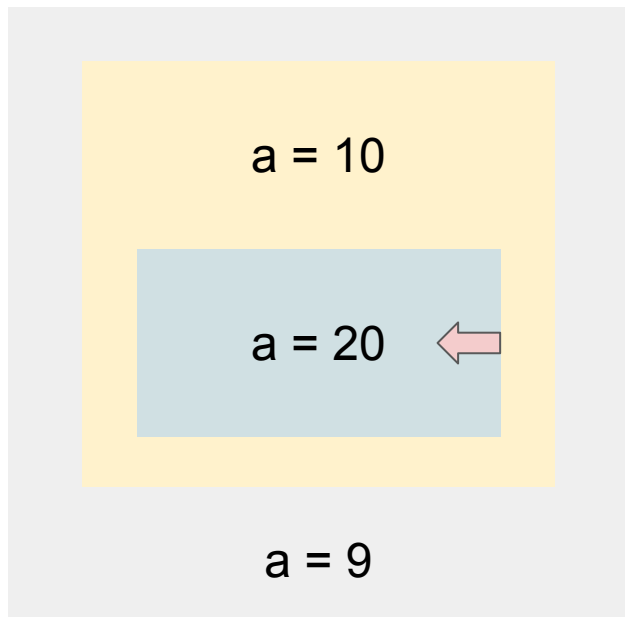
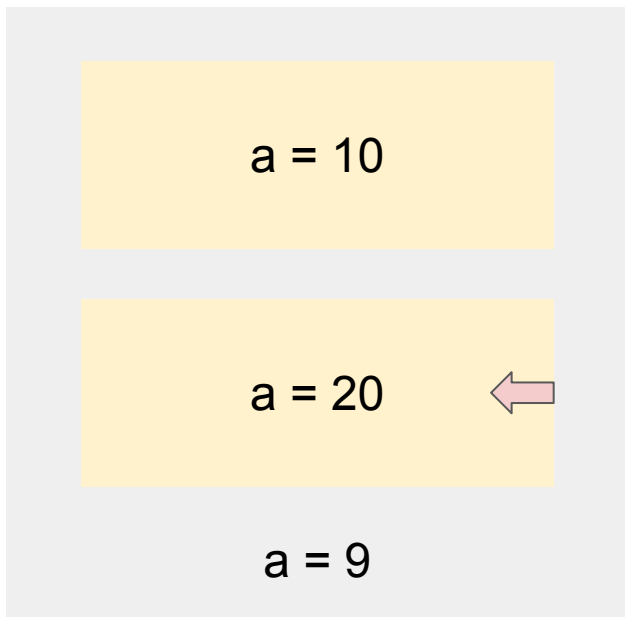
```
def add_list(shopping_list, item):  
    print(f"正在將加入清單...")  
    shopping_list.append(item)  
    print("加入成功！")
```



```
my_groceries = ["蘋果", "麵包"]  
add_list(my_groceries, "牛奶")
```

函數作用域 function scope

- function scope 可以讓程式在尋找變數時不那麼混亂，**由內往外找**。





函數作用域 function scope

- 請問以下答案是多少？

```
a = 9
def test1():
    a = 10
    def test2():
        a = 20
        print(a)
    test2()
```

```
test1()
```

- 請問以下答案是多少？

```
a = 9
def test1(a):
    a = 10
    def test2(a):
        a = 20
        print(a)
    test2(a)
```

```
test1(a)
```

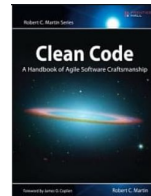


字串格式化 (String Formatting)

```
name = "小明"  
score = 95.56
```

- 傳統作法 (str 加法) 太長！容易出錯：
 - `greeting = "你好, " + name + "！你的期末考分數是 " + str(score) + " 分"`
 - `print(greeting)`
 - 可讀性太差，很容易出錯！
- 新作法：**F-String** (py36+)
 - **語法**：在字串的引號前加上一個 **f**，然後將變數或表達式直接用 **大括號 { }** 包起來。
 - `greeting = f"你好, {name}！你的期末考分數是 {score} 分"`
- F-String 小數點控制
 - `tmp = f"加權總分是: {score:.2f}"`
- F-String 數字補齊 (# d 代表整數, 04 代表總共4位數, 不足的補0)
 - `file_name = f"image_{file_id:04d}.jpg"`

補充：Clean Code - Coding Style



- 之後可以花時間看完下面兩篇，寫程式時盡力遵守 Coding Style：
 - [Google Python 風格指南 \(一\)](#)
 - [Google Python 風格指南 \(二\)](#)
- Reference：
 - <https://github.com/google/styleguide/blob/gh-pages/pyguide.md>

先看學過的語句就好，其他的等學到在回來看。



練習題

1. 給定 `list1 = [4, 6, 8, 24, 12, 2]`, 寫一個函數計算出最大值 `cal_max(par)`, 不可使用內建的 `max` BIF。
2. 寫一個函數, 每次輸入日期 `2021-01-05` 或 `01-05-2021` 就會 `print` 出:

您輸入的日期是: 2021 年 01 月 05 號

- **Hint: `split()` 用法**
- 要求以上兩種日期格式
 - 年-月-日
 - 月-日-年
- 使用字串格式化方法來 `print`

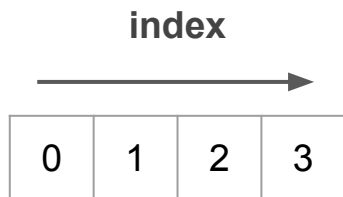
Python 套件介紹

Numpy 科學計算基礎

Numpy 資料型態

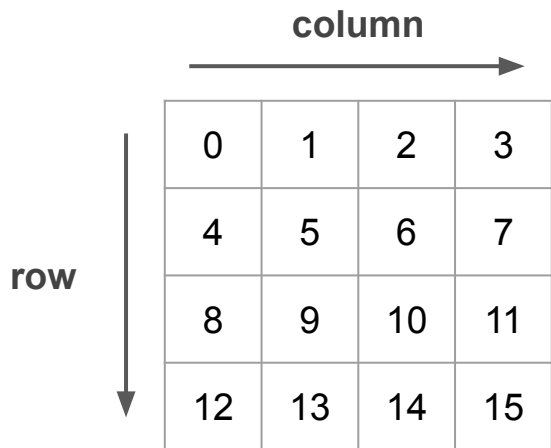
- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。

一維向量 vector



Shape : (index,)

二維矩陣 matrix



Shape : (row, column)

多維陣列 **array**

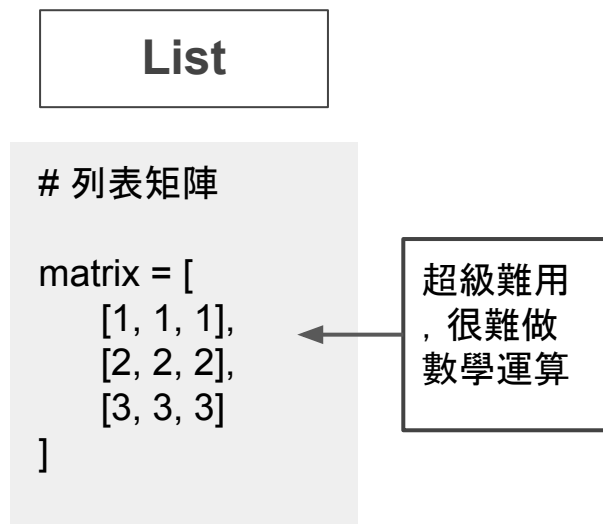
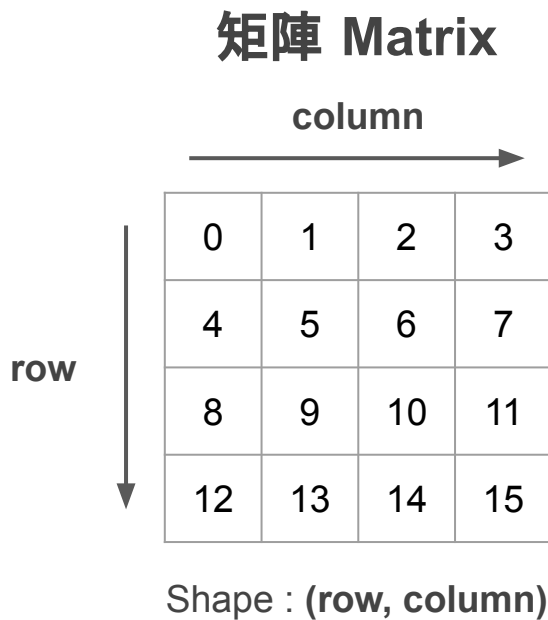
大家都是
ndarray



Shape : (a, b, c, d)

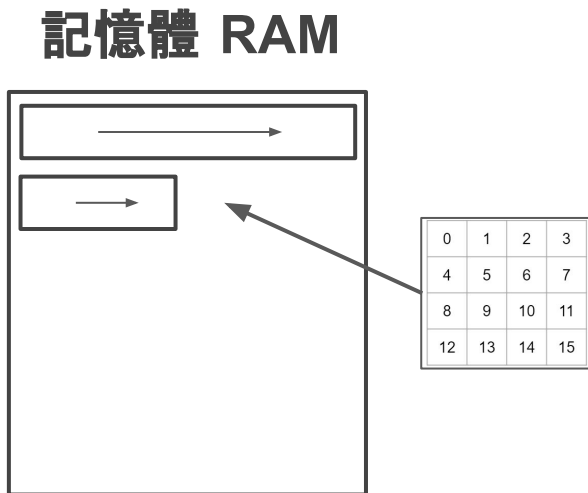
Numpy 資料型態

- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。



Numpy 資料型態

- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。



Numpy

C++ based

真正的矩陣

```
import numpy as np
```

```
matrix = np.array([  
    [1, 1, 1],  
    [2, 2, 2],  
    [3, 3, 3]  
])
```

使用 import
在 python
中的第三方
套件引入。

Numpy 資料型態

- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。

shape

```
matrix = np.array([
    [1, 1, 1],
    [2, 2, 2],
    [3, 3, 3]
])
```

```
# 使用 '!' 後面接方法
shape = matrix.shape
print(shape)
print(type(matrix))
```

方法有兩種：

- 放在前面
 - 類似 BIF
 - list(data)
 - np.array(data)
- 放在後面
 - 類似屬性
 - list1.append()
 - data.shape

其他常用創建方式

```
# 全 0 array
arr = np.zeros(10)
arr = np.zeros((2, 2))
```

```
print(arr)
```

```
# 全 1 array
arr = np.ones(10)
arr = np.ones((2, 2))
```

```
print(arr)
```

Numpy 資料型態

- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。

Slicing

```
matrix = np.array([
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8]
])

# Slicing [ ] 規則一樣
matrix[1, 1] # 4

matrix[2, 2] # 8
```

矩陣 Matrix

column →

row ↓

0	1	2
3	4	5
6	7	8

Numpy 更多的 Slicing

- Numpy 做 Slicing 時是非常自由的, 詳情看以下。

矩陣 Matrix

column →

row ↓

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

matrix[0, :] # 取 row

matrix[1:, 3] # 取 col

matrix[2:, :2] # 取區塊

多維陣列 **array**



Shape : (a, b, c, d)

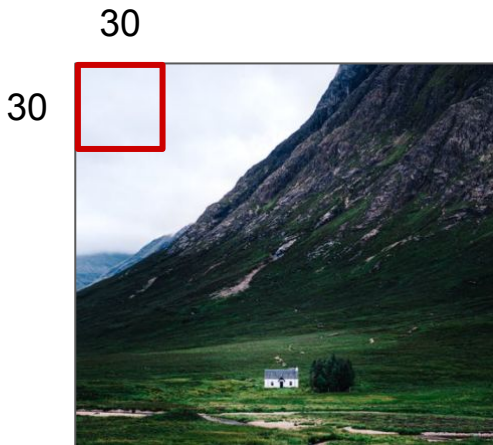
Numpy 更多的 Slicing

- Numpy 做 Slicing 時是非常自由的，詳情看以下。

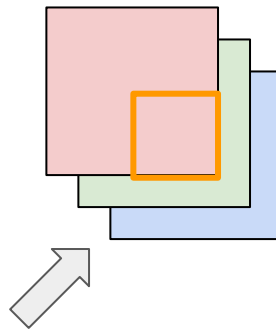
多維陣列 **array**



RGB Shape : (r, g, b)



Shape : (256, 256, 3)



怎麼取紅色區塊 ?

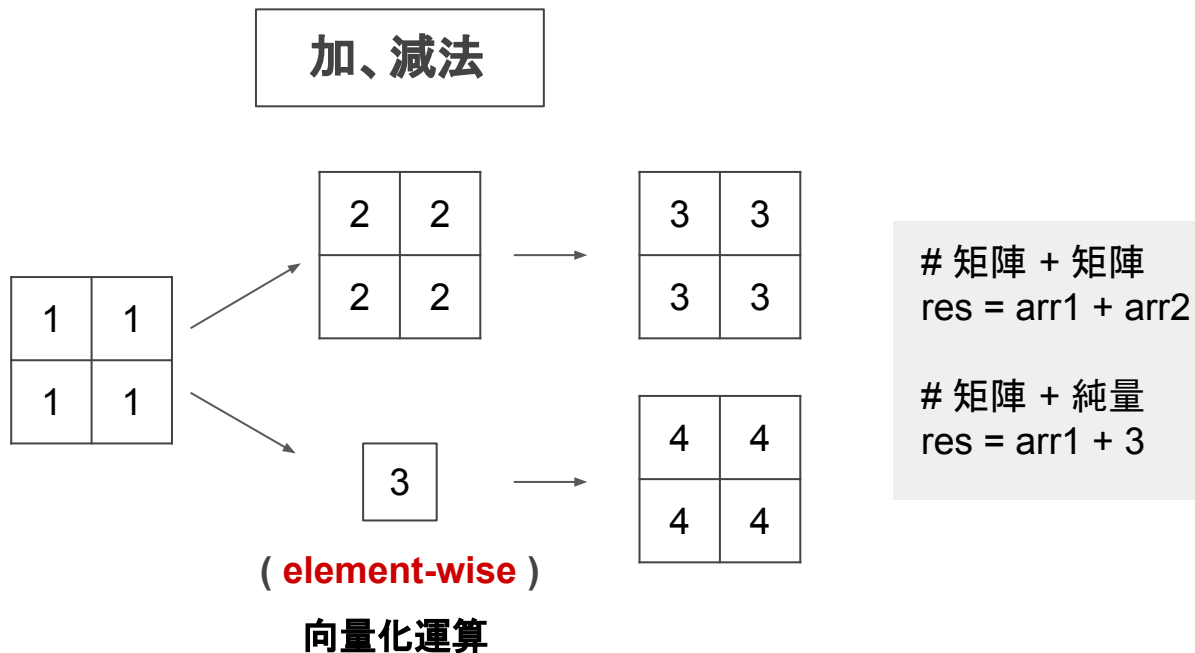
```
matrix[:30, :30, :]
```

怎麼取橘色區塊 ?

```
matrix[128:, 128:, 0]
```

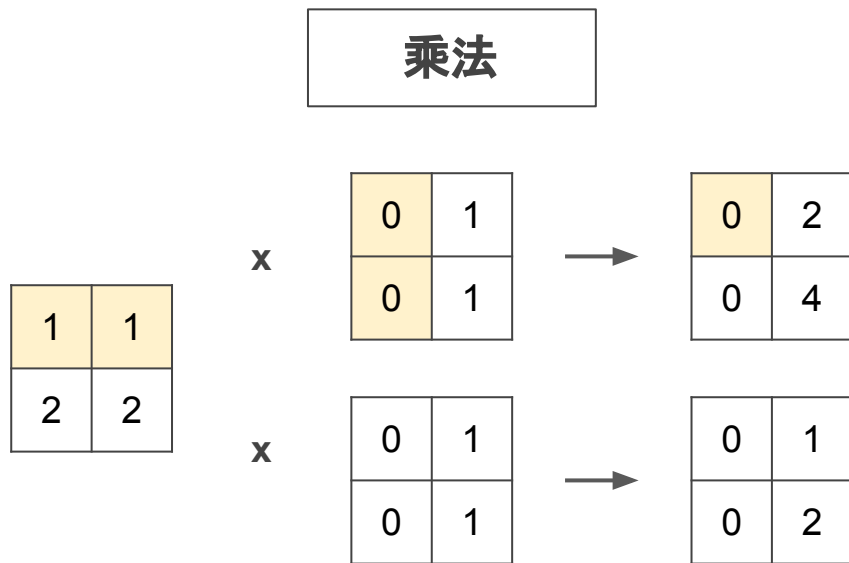
Numpy 基本矩陣運算

- 數學中矩陣有很多重要的運算功能都可以透過 numpy 來實現。



Numpy 基本矩陣運算

- 數學中矩陣有很多重要的運算功能都可以透過 numpy 來實現。



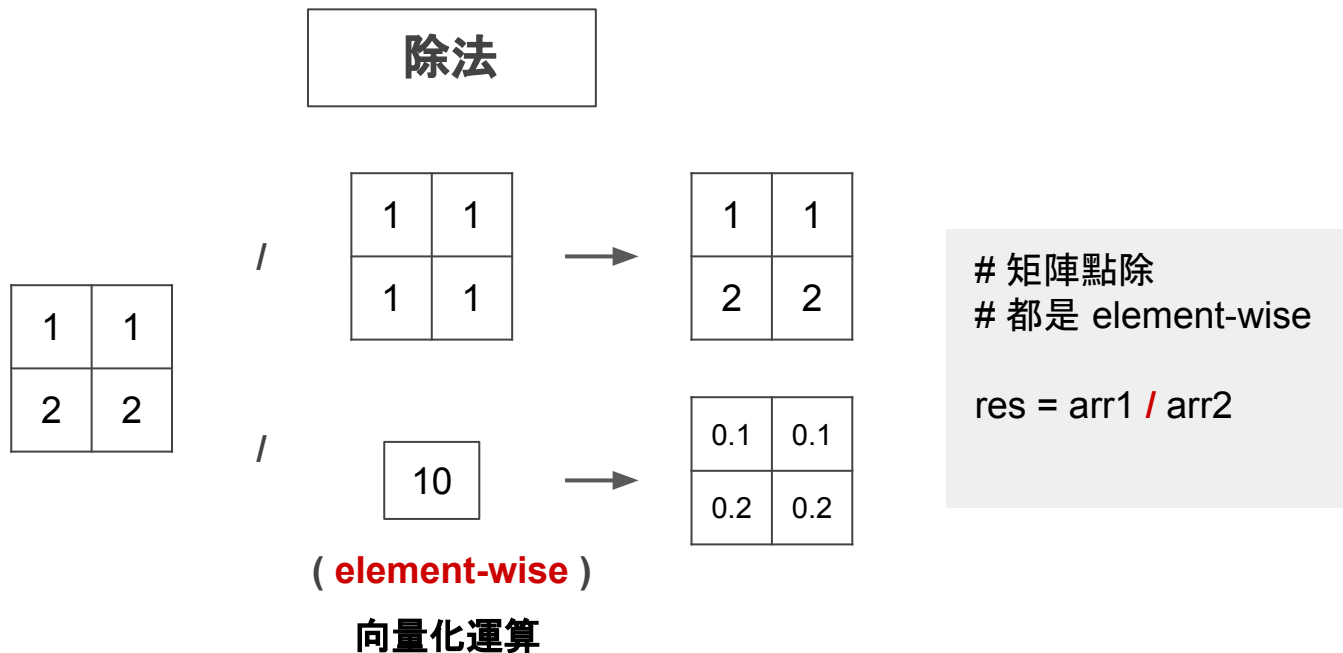
**特別注意不要用
錯**

```
# 矩陣相乘  
res = arr1.dot(arr2)
```

```
# 矩陣點乘  
res = arr1 * arr2
```

Numpy 基本矩陣運算

- 數學中矩陣有很多重要的運算功能都可以透過 numpy 來實現。



Numpy 常用的運算功能

- Numpy 除了在多維矩陣上好用外，在一般python 運算時也很有很多通用功能。
 - 與 List 和 Tuple 相容性很好。

np.sum

```
list1 = [1, 2, 3, 4]  
tup1 = (1, 2, 3, 4)
```

```
res = np.sum(list1)  
res = np.sum(tup1)
```

np.min

np.cos

np.max

np.exp

np.sin

np.log

(建議：要做數學運算時，優先想到 **numpy**)

Numpy 其他細節補充

- Numpy 的 array 只能用在數值上 (int, float), 其數值精度可以要求到很細。

32 精度 (同理 float)

```
arr = np.array([1, 2, 3], dtype='int32')
```

```
arr = np.array([1, 2, 3], dtype=np.int32)
```

64 精度 (同理 int)

```
arr = np.array([1, 2, 3], dtype='float64')
```

```
arr = np.array([1, 2, 3], dtype=np.float64)
```

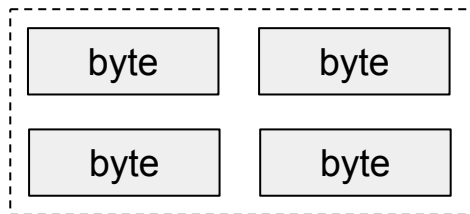
- 那什麼是精度, 在深度學習好像很常聽到?

Numpy 其他細節補充

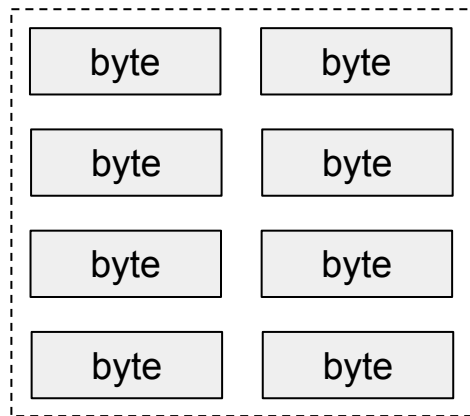


- 精度顧名思義代表要電腦去記一個數直到多精細。(ex. 1, 1.0, 1.000000000)
- 電腦可操作最小單位是byte, 而 1 byte 又包含 8 bits ???

32 精度



64 精度



- 深度學習或其他科學運算上, 經常要求到很精準的成果 (ex. 微分一次差一點, 但微個 100w 次就天差地遠...), **Label vs Data**

Numpy 其他細節補充

- 若是有需要與字串 (str) 一起儲存的需求改怎麼辦？
 - Pandas
 - 很像 excel / csv 檔案格式

Numpy

(array)

```
>>> a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```



Pandas

(dataframe)

Out[83]:

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

程式題 1-1

15 分鐘內請作答問答 1-1 到 1-3

1. 請引入 `numpy` 套件, 並且分別創建以下變數

a. 長度為 10 的全 0 向量。

b. `row` 和 `col` 分別為 2 和 5 的全 1 矩陣。

c.

0	1	2
3	4	5

2. 創建一個向量, 數值分別為 0, 1, 2, 3, ..., 100。

3. `np.random` 是 `numpy` 中一個很常被用來生成隨機數值使用的功能, 給定以下程式,

請計算出 `max`, `min`, `median`, `mean`, `std` 五個統計數值。

```
z = np.random.random(100) # 隨機數服從標準常態分布
```

程式題 1-2

1. 嘗試將以下的 array 進行Z標準化 (減平均數後再除標準差) :

0	1	2
3	4	5

2. 嘗試將以下的 array 各 row 和各 col 的平均數 (row 有 2 個 ans、col 有 3 個 ans) :

0	1	2
3	4	5

Hint: 查 axis 參數用法

3. 給定 A、B 向量, 計算 $(A+B)$ 點積 $(-A/2)$ 的結果 :

```
A = np.ones(3) * 1  
B = np.ones(3) * 2
```

程式題 1-3

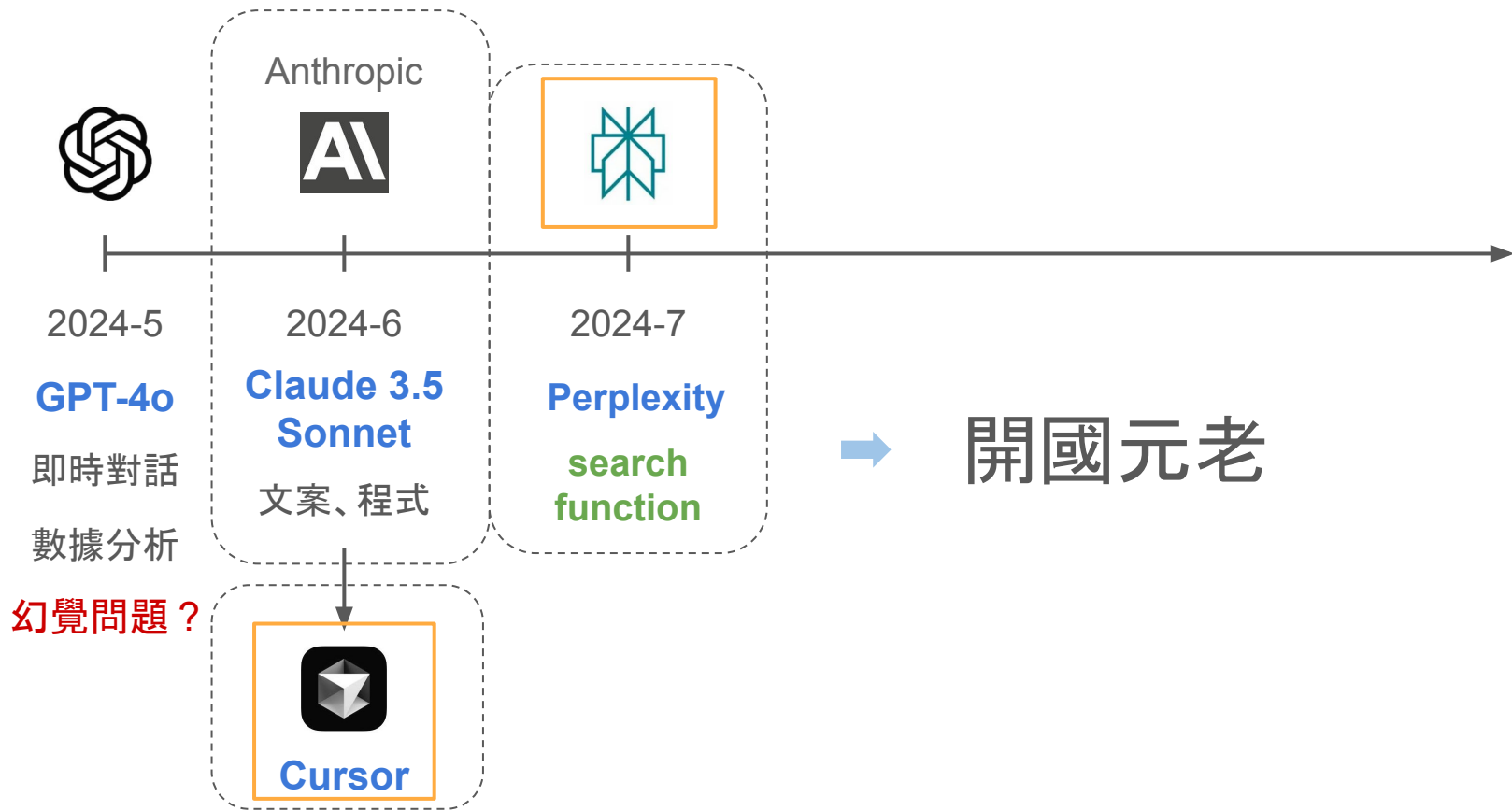
1. `np.random.randint` 可以從你給定的範圍中隨機產生N個整數, 請寫一個數字pk程式, 和 numpy 比賽數字大的贏, 用 `input` 要求輸入 (記得要用 `function` 去寫):

```
d = np.random.randint(0, 10, N) # 隨機數服從均勻分布
```

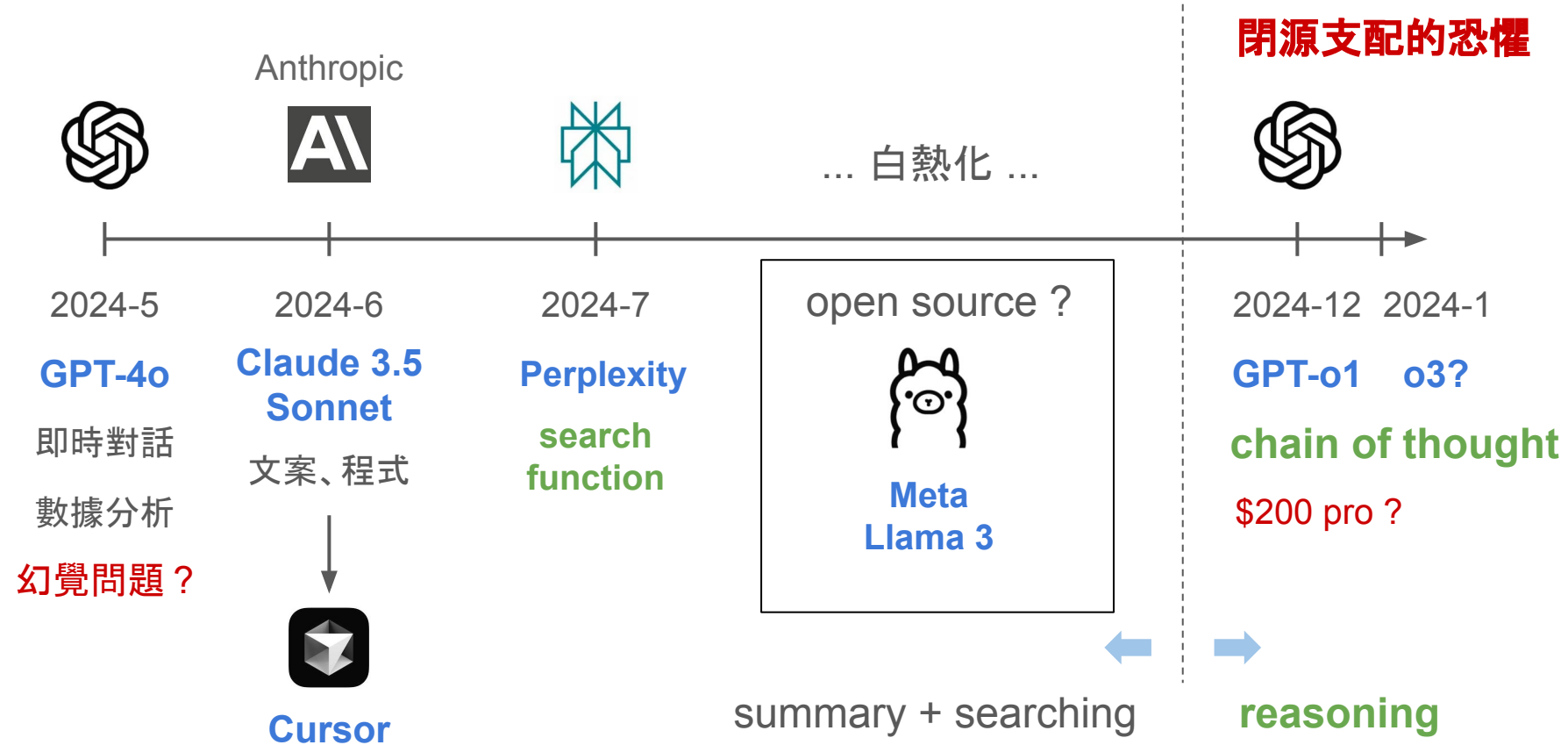
2. 給定 `list1 = [1, 2, 3, 5, 5, 5, 8, 9, 10]`, 請寫一個程式隨機抽取 `list1` 中的號碼, 一次抽取 3 個號碼: (記得要用 `function` 去寫)

補充資訊

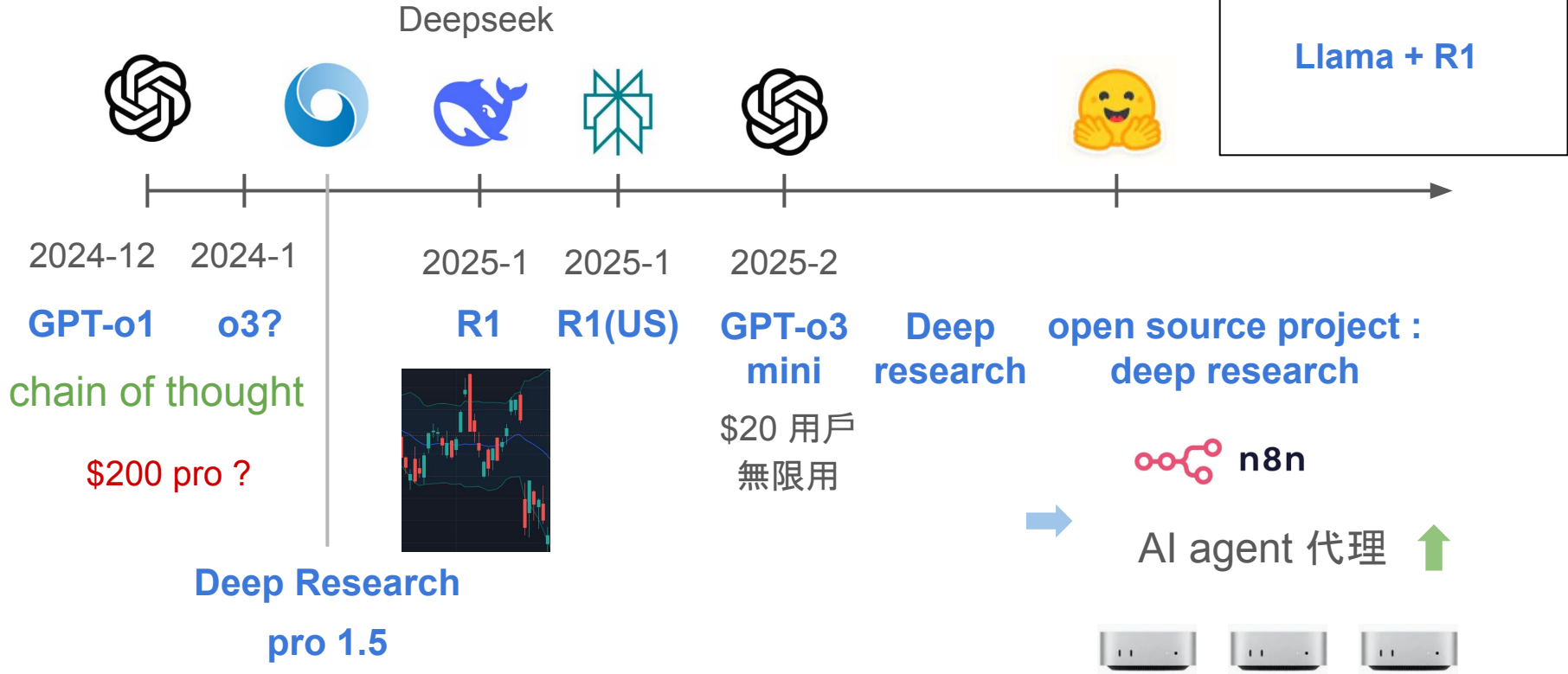
General AI tools - the new era



General AI tools - the new era



General AI tools - the new era



General AI tools - the new era

